

Winter of Code - 2025/26

Cyberlabs - Machine Learning Division (IIT (ISM) Dhanbad)

About

The winter of code 2025/26 was an online hackathon where we were supposed to make machine learning algorithms from scratch, we could use libraries like Numpy , Pandas and Matplotlib .

This `Readme.md` contains information about the algorithms implemented during the WOC -2025/26

Algorithms and models

The algorithms and models build were -

- ☐ Linear Regression
- ☐ Logistic Regression
- ☐ Polynomial Regression
- ☐ N - layer Neural Network
- ☐ K - Nearest Neighbours
- ☐ K Means Clustering
- ☐ Initializers - He, Xavier (NN)
- ☐ Optimizers - Gradient Descent and Adam (NN)

All of these algorithms were build from scratch

Folder and File structure

- `./datasets` - contains all of the datasets (train + test) provided for the algorithms.
 - `./outputs` - contains all of the outputs of test dataset after the predictions were made from the models.
 - `ml_library.py` - python library which have all of the algorithms and important function, all coded from scratch.
 - `ml_metrics_eval.py` - python library which contains the functions for evaluation metrics of models.
 - `*.ipynb` - python notebook where the all of the models were trained and tested on the dataset provided.
-

All of the algorithms were implemented using the class method from Object Oriented Programming (OOPs).

Models and Algorithms

1. Linear Regression

Linear Regression

Linear Regression is one of the most fundamental machine learning models. It tries to fit the maximum number of data points using a straight line, plane, or hyperplane based on the shape of the input features.

Here we create a Linear Regression class which has hyper-parameters like alpha (learning rate) and number of iterations.

```
class LinearRegression:
```

```
def __init__(self, alpha=0.01, iters=1000):

    self.alpha = alpha

    self.iters = iters

    self.weights = None

    self.bias = None

    self.J_history = []
```

Key Methods:

1. Cost Function: This function calculates the cost/loss function for the model. Our goal is to minimize this function by changing values of w and b .

$$J(w, b) = (1 / 2m) \times \sum \text{from } i = 1 \text{ to } m \text{ of } [(w^T x(i) + b) - y(i)]^2$$

```
def cost_function(self, x , y):

    m = x.shape[0]

    cost = 0

    for i in range(m):

        err = (np.dot(x[i], self.weights) + self.bias) - y[i]

        cost += err**2

    return cost/(2*m)
```

2. Gradient Calculation: Using this function to calculate the gradient dJ/dw and dJ/db .

$$dJ/dw = (1 / m) \times \sum \text{from } i = 1 \text{ to } m \text{ of } [(w \cdot x(i) + b - y(i)) \times x(i)]$$

$$dJ/db = (1 / m) \times \sum \text{from } i = 1 \text{ to } m \text{ of } [w \cdot x(i) + b - y(i)]$$

```
def gradient_calculation(self, x, y):  
  
    m = x.shape[0]  
  
    dj_dw = np.zeros(self.weights.shape)  
  
    dj_db = 0  
  
    for i in range(m):  
  
        err = (np.dot(x[i], self.weights) + self.bias) - y[i]  
  
        for j in range(len(self.weights)):  
  
            dj_dw[j] += err * x[i][j]  
  
        dj_db += err  
  
    dj_dw /= m  
  
    dj_db /= m  
  
    return dj_dw, dj_db
```

3. Gradient Descent: Updating the parameters w and b after each iteration to minimize the cost function.

```
def gradient_descent(self, x, y):  
  
    self.weights = np.zeros(x.shape[1])  
  
    self.bias = 0  
  
    for iter in range(self.iters):  
  
        dj_dw, dj_db = self.gradient_calculation(x, y)  
  
        self.weights -= self.alpha * dj_dw  
  
        self.bias -= self.alpha * dj_db  
  
        cost = self.cost_function(x, y)
```

```
self.J_history.append(cost)

if iter% math.ceil(self.iters / 10) == 0:

    print(f"Epoch {iter}: Cost {cost}")

    return self.weights, self.bias
```

4. Plotting Cost vs Iteration: Used for deciding the value of learning rate (alpha).

```
def plot_cost(self):

    plt.plot(self.J_history)

    plt.xlabel("No. of iterations")

    plt.ylabel("Cost")

    plt.title("Cost vs iterations")

    plt.show()
```

5. Evaluating the model: Using R2, Mean Square Error, Root Mean Square Error and Mean Absolute Error to evaluate the model's performance.

```
def evaluation_metrics(self, y_true, y_pred):

    r2 = r2_score(y_true, y_pred)

    mse = mean_squared_error(y_true, y_pred)

    rmse = root_mean_squared_error(y_true, y_pred)

    mae = mean_absolute_error(y_true, y_pred)

    print(f"R2 Score: {r2}")

    print(f"Mean Squared Error: {mse}")

    print(f"Root Mean Squared Error: {rmse}")
```

```
print(f"Mean Absolute Error: {mae}")
```

6. Model Summary: Prints the weight and bias of model along with the learning rate and number of iterations.

```
def _model_summary(self):  
  
    print("Model Summary:")  
  
    print(f"Weights: {self.weights}")  
  
    print(f"Bias: {self.bias}")  
  
    print(f"Learning Rate: {self.alpha}")  
  
    print(f"Iterations: {self.iters}")
```

2. Polynomial Regression

Polynomial Regression

Polynomial Regression extends Linear Regression by fitting a polynomial equation to the data. It can capture non-linear relationships between features and target variables.

This class inherits from LinearRegression and adds polynomial feature transformation capability.

```
class PolynomialRegression(LinearRegression):  
  
    def __init__(self, degree=2, alpha=0.01, iters=1000):  
  
        super().__init__(alpha, iters)  
  
        self.degree = degree
```

Key Methods:

1. Polynomial Features: Generates polynomial features for multiple variables including interaction terms.

```
def polynomial_features_multi(self, x, degree):  
  
    n_samples, n_features = x.shape  
  
    x_poly = [np.ones(n_samples)]  
  
    for d in range(1, degree + 1):  
  
        for feature_indices in self.generate_combinations(n_features, d):  
  
            feature = np.prod(x[:, feature_indices], axis=1)  
  
            x_poly.append(feature)  
  
    return np.column_stack(x_poly)
```

2. Generate Combinations: Creates all possible feature combinations for polynomial terms.

```
def generate_combinations(self, n_features, degree):  
  
    if degree == 1:  
  
        return [[i] for i in range(n_features)]  
  
    combinations = []  
  
    for i in range(n_features):  
  
        for sub_combination in self.generate_combinations(n_features, degree - 1):  
  
            if i <= sub_combination[0]:  
  
                combinations.append([i] + sub_combination)
```

```
return combinations
```

3. Fit: Transforms input features to polynomial features and fits the model.

```
def fit(self, x, y):  
  
    x_poly = self.polynomial_features_multi(x, self.degree)  
  
    self.weights, self.bias = self.gradient_descent(x_poly, y)  
  
    return self.weights, self.bias
```

4. Predict: Makes predictions using polynomial transformed features.

```
def predict(self, x):  
  
    x_poly = self.polynomial_features_multi(x, self.degree)  
  
    return np.dot(x_poly, self.weights) + self.bias
```

3. Logistic Regression

Logistic Regression

Logistic Regression is used for binary classification problems. It uses the sigmoid function to map predictions to probabilities between 0 and 1.

Creates a Logistic Regression classifier with configurable learning rate and iterations.

```
class LogisticRegression:  
  
    def __init__(self, alpha=0.01, iters=1000):  
  
        self.alpha = alpha
```



```
self.iters = iters

self.weights = None

self.bias = None

self.J_history = []
```

Key Methods:

1. Sigmoid Function: Transforms linear predictions to probabilities.

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

```
def sigmoid(self, z):

    return 1 / (1 + np.exp(-z))
```

2. Cost Function: Binary cross-entropy loss for classification.

$$J(w, b) = -(1/m) \times \sum [y(i)\log(f(x(i))) + (1-y(i))\log(1-f(x(i)))]$$

```
def cost_function(self, x, y):

    m = len(x)

    cost = 0

    for i in range(m):

        z = np.dot(x[i], self.weights) + self.bias

        f_wb = self.sigmoid(z)

        cost += - (y[i] * np.log(f_wb) + (1 - y[i]) * np.log(1 - f_wb))

    return cost/m
```

3. Gradient Calculation: Computes gradients for logistic regression.

```
def gradient_calculation(self, x, y):  
  
    m = len(x)  
  
    dj_dw = np.zeros(self.weights.shape)  
  
    dj_db = 0  
  
    for i in range(m):  
  
        z = np.dot(x[i], self.weights) + self.bias  
  
        f_wb = self.sigmoid(z)  
  
        err = f_wb - y[i]  
  
        for j in range(len(self.weights)):  
  
            dj_dw[j] += err * x[i][j]  
  
        dj_db += err  
  
    dj_dw /= m  
  
    dj_db /= m  
  
    return dj_dw, dj_db
```

4. Predict: Returns binary predictions using 0.5 threshold.

```
def predict(self, X):  
  
    z = np.dot(X, self.weights) + self.bias  
  
    y_pred_proba = self.sigmoid(z)  
  
    return (y_pred_proba >= 0.5).astype(int)
```

5. Probability: Returns prediction probabilities.

```
def probability(self, X):  
  
    z = np.dot(X, self.weights) + self.bias  
  
    return self.sigmoid(z)
```

6. Evaluation Metrics: Provides accuracy, precision, recall, F1 score and confusion matrix.

```
def evaluation_metrics(self, y_true, y_pred):  
  
    accuracy = accuracy_score(y_true, y_pred)  
  
    precision = precision_score(y_true, y_pred)  
  
    recall = recall_score(y_true, y_pred)  
  
    f1 = f1_score(y_true, y_pred)  
  
    cm = confusion_matrix(y_true, y_pred)  
  
    print(f"Accuracy: {accuracy}")  
  
    print(f"Precision: {precision}")  
  
    print(f"Recall: {recall}")  
  
    print(f"F1 Score: {f1}")  
  
    print(f"Confusion Matrix:\n{cm}")
```

4. Neural Network

Neural Network

A flexible neural network implementation supporting multiple layers, activation functions, optimizers, and initialization methods.

Creates a neural network with configurable learning rate, epochs, loss function, weight initialization, and optimizer.

```
class NeuralNetwork:

    def __init__(self, learning_rate=0.01, epochs=1000, loss_function='mse',
                  initialization='random', optimizer='adam', beta1=0.9,
                  beta2=0.999, epsilon=1e-8):

        self.learning_rate = learning_rate

        self.epochs = epochs

        self.loss_function = loss_function

        self.initialization = initialization

        self.optimizer = optimizer
```

Activation Functions:

1. Sigmoid: Used for binary classification output layers.

```
def sigmoid(self, z):

    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(self, z):

    a = self.sigmoid(z)

    return a * (1 - a)
```

2. ReLU: Commonly used activation function for hidden layers.

```
def relu(self, z):  
  
    return np.maximum(0, z)  
  
def relu_derivative(self, z):  
  
    return np.where(z > 0, 1, 0)
```

3. Linear: Identity function for regression problems.

```
def linear(self, z):  
  
    return z  
  
def linear_derivative(self, z):  
  
    return np.ones_like(z)
```

4. Softmax: Used for multi-class classification output layers.

```
def softmax(self, z):  
  
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))  
  
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)
```

Model Architecture:

Sequential Model: Creates a sequential neural network model.

```
def Sequential(self, *layers):
```

```
return {  
  
    'layers': layers,  
  
    'parameters': {},  
  
}
```

Dense Layer: Adds a fully connected layer with specified units and activation.

```
def Dense(self, units, activation='relu'):  
  
    return {  
  
        'units': units,  
  
        'activation': activation  
  
    }
```

Weight Initialization:

1. Random Initialization: Standard random initialization scaled by 0.01.

```
def initialize_parameters(self, model, input_dim):  
  
    parameters = {}  
  
    layer_dims = [input_dim]  
  
    for layer in model['layers']:  
  
        layer_dims.append(layer['units'])  
  
    L = len(layer_dims)  
  
    for l in range(1, L):  
  
        parameters['w' + str(l)] = np.random.randn(layer_dims[l-1], layer_dims[l])
```

```
parameters['b' + str(l)] = np.zeros((1, layer_dims[l]))

model['parameters'] = parameters

return model
```

2. He Initialization: Ideal for ReLU activation functions.

```
def initialize_parameters_he(self, model, input_dim):

# Weight initialization: w * sqrt(2 / layer_dims[l-1])
```

3. Xavier Initialization: Ideal for sigmoid and tanh activations.

```
def initialize_parameters_xavier(self, model, input_dim):

# Weight initialization: w * sqrt(1 / layer_dims[l-1])
```

Forward and Backward Propagation:

1. Forward Propagation: Computes predictions through the network layers.

```
def forward_propagation(self, X, model):

parameters = model['parameters']

caches = {'a0': X}

L = len(model['layers'])

for l in range(1, L + 1):

z = np.dot(caches['a' + str(l-1)], parameters['w' + str(l)]) + parameters['

# Apply activation function based on layer configuration

caches['z' + str(l)] = z
```

```
    caches['a' + str(l)] = a

    return caches[f'a{L}'], caches
```

2. Backward Propagation: Computes gradients for all parameters using the chain rule.

```
def backward_propagation(self, X, y, model, caches):

    m = X.shape[0]

    parameters = model['parameters']

    grads = {}

    L = len(model['layers'])

    # Computes gradients starting from output layer

    # Propagates backwards through all layers

    return grads
```

Optimizers:

Adam Optimizer: Adaptive learning rate optimization with momentum.

```
def update_parameters(self, model, grads):

    if self.optimizer == 'adam':

        self.t += 1

        # Update first moment (momentum)

        # Update second moment (RMSprop)

        # Compute bias-corrected estimates
```



```
# Update parameters
```

Training:

Fit Method: Trains the neural network on provided data.

```
def fit(self, X, y, model):  
  
    input_dim = X.shape[1]  
  
    # Initialize parameters based on chosen method  
  
    # Initialize optimizer parameters if using Adam  
  
    for epoch in range(self.epochs):  
  
        y_pred, caches = self.forward_propagation(X, model)  
  
        grads = self.backward_propagation(X, y, model, caches)  
  
        model = self.update_parameters(model, grads)  
  
        # Calculate and store loss  
  
        if epoch % math.ceil(self.epochs / 10) == 0:  
  
            print(f"Epoch {epoch}: Loss {loss}")  
  
    return model
```

Evaluation Metrics: Provides metrics based on the loss function (regression or classification).

```
def evaluation_metrics(self, y_true, y_pred):  
  
    if self.loss_function == 'mse':  
  
        # Returns R2, MSE, RMSE, MAE
```

```
elif self.loss_function in ['binary_cross_entropy', 'categorical_cross_ent  
  
# Returns accuracy, precision, recall, F1
```

5. Decision Tree Classifier

Decision Tree Classifier

Decision Trees are non-parametric supervised learning methods used for classification. They create a tree structure where each internal node represents a test on a feature, branches represent outcomes, and leaf nodes represent class labels.

Creates a Decision Tree classifier with configurable minimum samples for split, maximum depth, and splitting criterion (gini or entropy).

```
class DecisionTreeClassifier:  
  
    def __init__(self, min_sample_split=2, max_depth=2, mode='gini'):  
  
        self.root = None  
  
        self.min_sample_split = min_sample_split  
  
        self.max_depth = max_depth  
  
        self.mode = mode
```

Key Methods:

1. Build Tree: Recursively constructs the decision tree using the training data.

```
def build_tree(self, dataset, curr_depth=0):  
  
    X, y = dataset[:, :-1], dataset[:, -1]
```

```
m, n = X.shape

if m >= self.min_sample_split and curr_depth <= self.max_depth:

    best_split = self.get_best_split(dataset, m, n)

    if best_split['info_gain'] > 0:

        left_subtree = self.build_tree(best_split['left_split'], curr_depth+1)

        right_subtree = self.build_tree(best_split["right_split"], curr_depth+1)

    return Node(feature_index, threshold, left, right, info_gain)

leaf_value = self.calculate_leaf_value(y)

return Node(value=leaf_value)
```

2. Get Best Split: Finds the optimal feature and threshold for splitting the dataset.

```
def get_best_split(self, dataset, num_samples, num_features):

    best_split = {}

    max_info_gain = -float('inf')

    for feature_index in range(num_features):

        feature_values = dataset[:,feature_index]

        threshold_vals = np.unique(feature_values)

        for threshold in threshold_vals:

            left_split, right_split = self.split(dataset, feature_index, threshold)

            # Calculate information gain and update best split

    return best_split
```

3. Information Gain: Calculates the reduction in impurity from a split.

Information Gain = Parent Impurity - Weighted Average of Child Impurities

```
def information_gain(self, parent, l_child, r_child):

    w_left = len(l_child)/len(parent)

    w_right = len(r_child)/len(parent)

    if self.mode == 'gini':

        gain = self.gini(parent) - (w_left * self.gini(l_child) + w_right * self.gi

    elif self.mode == 'entropy':

        gain = self.entropy(parent) - (w_left * self.entropy(l_child) + w_right * s

    return gain
```

4. Gini Impurity: Measures the probability of incorrect classification.

Gini = $1 - \sum(p_i^2)$ where p_i is the probability of class i

```
def gini(self, y):

    class_labels = np.unique(y)

    gini = 0

    for cls in class_labels:

        p_cls = len(y[y==cls])/ len(y)

        gini += p_cls**2

    return 1-gini
```

5. Entropy: Measures the amount of information disorder in the data.

Entropy = $-\sum(p_i \times \log_2(p_i))$

```
def entropy(self, y):  
  
    class_labels = np.unique(y)  
  
    entropy = 0  
  
    for cls in class_labels:  
  
        p_cls = len(y[y==cls])/len(y)  
  
        entropy += -p_cls * np.log2(p_cls)  
  
    return entropy
```

6. Predict: Makes predictions by traversing the tree for each sample.

```
def predict(self, X):  
  
    predictions = [self.make_prediction(x, self.root) for x in X]  
  
    return predictions  
  
def make_prediction(self, x, tree):  
  
    if tree.value != None:  
  
        return tree.value  
  
    feature_val = x[tree.feature_index]  
  
    if feature_val <= tree.threshold:  
  
        return self.make_prediction(x, tree.left)  
  
    else:  
  
        return self.make_prediction(x, tree.right)
```

7. Evaluation Metrics: Provides accuracy, precision, recall, F1 score and confusion matrix.

```
def evaluation_metrics(self, y_true, y_pred):  
  
    accuracy = accuracy_score(y_true, y_pred)  
  
    precision = precision_score(y_true, y_pred)  
  
    recall = recall_score(y_true, y_pred)  
  
    f1 = f1_score(y_true, y_pred)  
  
    cm = confusion_matrix(y_true, y_pred)
```

6. K-Nearest Neighbors (KNN)

K-Nearest Neighbors

KNN is a simple, instance-based learning algorithm that classifies data points based on the classes of their k nearest neighbors in the feature space.

Creates a KNN classifier with configurable number of neighbors (k).

```
class KNN:  
  
    def __init__(self, k=3):  
  
        self.k = k
```

Key Methods:

1. Fit: Stores the training data for later use in predictions.

```
def fit(self, X, y):  
  
    self.X_train = X  
  
    self.y_train = y
```

2. Distance: Calculates Euclidean distance between two points.

$\text{distance} = \sqrt{\sum (a_i - b_i)^2}$

```
def distance(self, a, b):  
  
    return np.sqrt(np.sum(np.square(a-b)))
```

3. Predict: Classifies new data points based on majority vote of k nearest neighbors.

```
def predict(self, X):  
  
    pred = []  
  
    for x_pred in X:  
  
        distances_train = [self.distance(x_pred, x_train) for x_train in self.X_train]  
  
        k_indices = np.argsort(distances_train)[:self.k]  
  
        k_labels = [self.y_train[i] for i in k_indices]  
  
        pred.append(max(set(k_labels), key=k_labels.count))  
  
    return np.array(pred)
```

4. Evaluation Metrics: Provides accuracy, precision, recall, F1 score and confusion matrix visualization.

```
def evaluation_metrics(self, y_true, y_pred):
```

```
accuracy = accuracy_score(y_true, y_pred)

precision = precision_score(y_true, y_pred)

recall = recall_score(y_true, y_pred)

f1 = f1_score(y_true, y_pred)

cm = confusion_matrix(y_true, y_pred)

# Visualizes confusion matrix
```

7. K-Means Clustering

K-Means Clustering

K-Means is an unsupervised learning algorithm that partitions data into k clusters by iteratively assigning points to the nearest centroid and updating centroids based on cluster means.

Creates a K-Means clustering model with configurable number of clusters and maximum iterations.

```
class KMeansClustering:

    def __init__(self, k=3, max_iters=100):

        self.k = k

        self.max_iters = max_iters

        self.centroids = None
```

Key Methods:

1. Initialize Centroids: Randomly selects k data points as initial centroids.


```
def initialize_centroids(self, X):  
  
    random_indices = np.random.choice(X.shape[0], self.k, replace=False)  
  
    self.centroids = X[random_indices]
```

2. Assign Clusters: Assigns each data point to the nearest centroid.

```
def assign_clusters(self, X):  
  
    distances = np.array([[np.linalg.norm(x - centroid) for centroid in self.c  
  
    return np.argmin(distances, axis=1)
```

3. Update Centroids: Recalculates centroids as the mean of all points in each cluster.

```
def update_centroids(self, X, labels):  
  
    new_centroids = np.array([X[labels == k].mean(axis=0) for k in range(self.  
  
    return new_centroids
```

4. Fit: Trains the K-Means model by iteratively assigning clusters and updating centroids until convergence.

```
def fit(self, X):  
  
    self.initialize_centroids(X)  
  
    for _ in range(self.max_iters):  
  
        labels = self.assign_clusters(X)  
  
        new_centroids = self.update_centroids(X, labels)
```

```
if np.linalg.norm(self.centroids - new_centroids) == 0:

    break

self.centroids = new_centroids
```

5. Predict: Assigns cluster labels to new data points.

```
def predict(self, X):

    return self.assign_clusters(X)
```

6. Plot Clusters: Visualizes the clustering results and centroids.

```
def plot_clusters(self, X):

    labels = self.predict(X)

    plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')

    plt.scatter(self.centroids[:, 0], self.centroids[:, 1], s=300, c='red', mar

    plt.title("K-Means Clustering")

    plt.show()
```

7. Elbow Method: Helps determine the optimal number of clusters by plotting WCSS (Within-Cluster Sum of Squares).

```
def plot_elbow_method(self, X, max_k=10):

    wcss = []

    for k in range(1, max_k + 1):

        kmeans = KMeansClustering(k=k)

        kmeans.fit(X)
```

```
wcss.append(sum(np.min([[np.linalg.norm(x - centroid) for centroid in kmean

plt.plot(range(1, max_k + 1), wcss, marker='o')

plt.title("Elbow Method")

plt.xlabel("Number of clusters (k)")

plt.ylabel("WCSS")

plt.show()
```

Helpful functions in ml_library.py

1. Train-Test Split

train_test_split

The train-test split function divides a dataset into training and testing subsets. This is crucial for evaluating model performance on unseen data and preventing overfitting.

Purpose:

- **Training set:** Used to train the model and learn patterns
- **Testing set:** Used to evaluate model performance on new, unseen data

Parameters:

- **x** : Feature matrix
- **y** : Target variable
- **test_size** : Proportion of dataset to include in test split (default: 0.2 or 20%)

How it works:

1. Calculates the number of test samples based on `test_size`
2. Creates random indices for the entire dataset
3. Shuffles the indices to ensure random sampling
4. Splits indices into test and train sets
5. Uses indices to partition X and y

Code:

```
def train_test_split(X, y, test_size=0.2):  
  
    n_samples = len(X)  
  
    n_test = int(test_size * n_samples)  
  
    indices = np.arange(n_samples)  
  
    np.random.shuffle(indices)  
  
    test_indices = indices[:n_test]  
  
    train_indices = indices[n_test:]  
  
    X_train = X[train_indices]  
  
    X_test = X[test_indices]  
  
    y_train = y[train_indices]  
  
    y_test = y[test_indices]  
  
    return X_train, X_test, y_train, y_test
```

Example Usage:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# 80% for training, 20% for testing
```

2. Z-Score Normalization

z_score_normalization

Z-score normalization (also called standardization) transforms features to have zero mean and unit variance. This is important when features have different scales and ranges.

Purpose:

- Brings all features to the same scale
- Prevents features with larger ranges from dominating the model
- Helps gradient descent converge faster
- Required for algorithms sensitive to feature scales (KNN, Neural Networks, SVM)

Mathematical Formula:

$$z = (x - \mu) / \sigma$$

Where:

- x = original value
- μ = mean of the feature
- σ = standard deviation of the feature
- z = normalized value

How it works:

1. Calculates the mean for each feature across all samples
2. Calculates the standard deviation for each feature
3. Subtracts mean and divides by standard deviation for each feature

Code:

```
def z_score_normalization(X):  
  
    mean = np.mean(X, axis=0)  
  
    std = np.std(X, axis=0, ddof=0) # ddof=0 for population std, used in sklearn  
  
    X_normalized = (X - mean) / std  
  
    return X_normalized
```

Example Usage:

```
X_normalized = z_score_normalization(X_train)  
  
# Each feature now has mean=0 and std=1
```

Note:

- `ddof=0` uses population standard deviation (divides by N) instead of sample standard deviation (divides by N-1)
- This matches sklearn's `StandardScaler` behavior

3. One-Hot Encoding

`one_hot_encode`

One-hot encoding converts categorical integer labels into binary vectors. Each category becomes a separate binary feature where 1 indicates presence and 0 indicates absence.

Purpose:

- Converts categorical labels to a format suitable for neural networks
- Required for multi-class classification with softmax activation
- Prevents the model from interpreting ordinal relationships between classes

Example Transformation:

Original labels: [0, 1, 2, 1, 0]

One-hot encoded:

`[[1, 0, 0], # Class 0`

`[0, 1, 0], # Class 1`

`[0, 0, 1], # Class 2`

`[0, 1, 0], # Class 1`

`[1, 0, 0]] # Class 0`

How it works:

1. Converts labels to integers and flattens to 1D array
2. Determines number of classes (max value + 1)
3. Creates an identity matrix of size (num_classes × num_classes)

4. Uses label values as indices to select appropriate rows

Code:

```
def one_hot_encode(y):  
  
    y = y.astype(int).flatten()  
  
    num_class = np.max(y) + 1  
  
    return np.eye(num_class)[y]
```

Example Usage:

```
y_labels = np.array([0, 1, 2, 1, 0])  
  
y_encoded = one_hot_encode(y_labels)  
  
# Returns: [[1,0,0], [0,1,0], [0,0,1], [0,1,0], [1,0,0]]
```

Note:

- Assumes labels start from 0 and are consecutive integers
- Uses numpy's identity matrix (`np.eye`) for efficient encoding

4. Principal Component Analysis (PCA)

pca

PCA is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving as much variance as possible.

Purpose:

- **Reduces dimensionality:** Decreases number of features while retaining important information
- **Removes noise:** Filters out less important variations in data
- **Visualization:** Makes high-dimensional data visualizable (reducing to 2D or 3D)
- **Speeds up training:** Fewer features mean faster model training
- **Addresses multicollinearity:** Creates uncorrelated features

Mathematical Concept:

PCA finds new axes (principal components) along which data has maximum variance. These components are:

- Orthogonal (perpendicular) to each other
- Ordered by variance (1st component has highest variance)
- Linear combinations of original features

How it works:

1. **Center the data:** Subtracts mean to center data around origin
2. **Compute covariance matrix:** Measures how features vary together
3. **Calculate eigenvalues and eigenvectors:**
 - Eigenvalues represent variance along each component
 - Eigenvectors represent directions of principal components
4. **Sort by eigenvalues:** Orders components by importance (descending)
5. **Select top n components:** Keeps only the most important components
6. **Project data:** Transforms original data onto new axes

Code:

```
def pca(x, n):  
  
    # Step 1: Center the data  
  
    x_mean = x - x.mean(axis=0)  
  
    # Step 2: Compute covariance matrix  
  
    cov_matrix = np.cov(x_mean, rowvar=False)  
  
    # Step 3: Get eigenvalues and eigenvectors  
  
    eigenval, eigenvectors = np.linalg.eigh(cov_matrix)  
  
    # Step 4: Sort by eigenvalues (descending)  
  
    sorted_indices = np.argsort(eigenval)[::-1]  
  
    # Step 5: Select top n eigenvectors  
  
    max_eigen_vectors = eigenvectors[:, sorted_indices[:n]]  
  
    # Step 6: Project data onto new axes  
  
    projected_value = np.dot(x_mean, max_eigen_vectors)  
  
    return projected_value
```

Parameters:

- `x` : Original feature matrix (m samples $\times$ d features)
- `n` : Number of principal components to keep

Returns:

- Transformed data with reduced dimensions (m samples $\times$ n components)

Example Usage:

```
# Reduce 10 features to 3 principal components

X_reduced = pca(X_train, n=3)


# Reduce to 2D for visualization

X_2d = pca(X_train, n=2)

plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y_train)
```

When to use PCA:

- Dataset has many features (high dimensionality)
- Features are correlated
- Want to visualize high-dimensional data
- Need to speed up training without losing much information
- Want to remove noise from data

Important Notes:

- Should be applied **after** train-test split
- Should be fit on training data only, then applied to test data
- Works best when features are on similar scales (use normalization first)
- `np.linalg.eigh` is used for symmetric matrices (covariance is always symmetric)

Evaluation functions of `ml_eval.py`

Regression Metrics

These metrics are used to evaluate the performance of regression models that predict continuous values.

1. R^2 Score (Coefficient of Determination)

r2_score

R^2 score measures how well the model explains the variance in the target variable. It indicates the proportion of variance in the dependent variable that is predictable from the independent variables.

Range:

Best value: 1.0 (perfect predictions)

Baseline: 0.0 (model performs as well as predicting the mean)

Negative values: Model performs worse than predicting the mean

Mathematical Formula:

$$R^2 = 1 - (SS_{\text{residual}} / SS_{\text{total}})$$

Where:

$$SS_{\text{residual}} = \sum (y_{\text{true}} - y_{\text{pred}})^2 \text{ (Sum of squared residuals)}$$

$$SS_{\text{total}} = \sum (y_{\text{true}} - \bar{y})^2 \text{ (Total sum of squares)}$$

$\bar{y}$ = mean of true values

Interpretation:

$R^2 = 0.85$: Model explains 85% of the variance in the data

$R^2 = 0.50$: Model explains 50% of the variance

$R^2 = 0.00$: Model is no better than predicting the mean

Code:

```
pythondef r2_score(y_true, y_pred):  
  
    ss_residual = np.sum((y_true - y_pred) ** 2)  
  
    ss_total = np.sum((y_true - np.mean(y_true)) ** 2)  
  
    r2 = 1 - (ss_residual / ss_total)  
  
    return r2
```

Example Usage:

```
pythony_true = np.array([3, -0.5, 2, 7])  
  
y_pred = np.array([2.5, 0.0, 2, 8])  
  
score = r2_score(y_true, y_pred)
```

Higher is better (max = 1.0)

2. Mean Squared Error (MSE)

mean_squared_error

MSE measures the average squared difference between predicted and actual values. It heavily penalizes large errors due to squaring.

Range:

Best value: 0.0 (perfect predictions)

Range: $[0, \infty)$

Lower is better

Mathematical Formula:

$$\text{MSE} = (1/n) \times \sum (y_{\text{true}} - y_{\text{pred}})^2$$

Characteristics:

Sensitive to outliers: Large errors are penalized more due to squaring

Same units as target squared: If predicting price in dollars, MSE is in dollars²

Differentiable: Useful for gradient-based optimization

Code:

```
pythondef mean_squared_error(y_true, y_pred):
```

```
mse = np.mean((y_true - y_pred) ** 2)
```

```
return mse
```

Example Usage:

```
pythony_true = np.array([3, -0.5, 2, 7])
```

```
y_pred = np.array([2.5, 0.0, 2, 8])
```

```
mse = mean_squared_error(y_true, y_pred)
```

Lower is better (min = 0.0)

3. Root Mean Squared Error (RMSE)

root_mean_squared_error

RMSE is the square root of MSE, making it interpretable in the same units as the target variable.

Range:

Best value: 0.0 (perfect predictions)

Range: $[0, \infty)$

Lower is better

Mathematical Formula:

$$\text{RMSE} = \sqrt{(\text{MSE})} = \sqrt{[(1/n) \times \sum(y_{\text{true}} - y_{\text{pred}})^2]}$$

Advantages over MSE:

Same units as target: If predicting price in dollars, RMSE is also in dollars

More interpretable: Easier to understand error magnitude

Still penalizes large errors: Due to squaring before taking root

Code:

```
pythondef root_mean_squared_error(y_true, y_pred):  
  
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))  
  
    return rmse
```

Example Usage:

```
pythony_true = np.array([100, 150, 200, 250])  
  
y_pred = np.array([110, 145, 205, 240])  
  
rmse = root_mean_squared_error(y_true, y_pred)
```

RMSE ≈ 8.66 (in same units as predictions)

4. Mean Absolute Error (MAE)

mean_absolute_error

MAE measures the average absolute difference between predicted and actual values. Unlike MSE, it treats all errors equally.

Range:

Best value: 0.0 (perfect predictions)

Range: $[0, \infty)$

Lower is better

Mathematical Formula:

$$\text{MAE} = (1/n) \times \sum |y_{\text{true}} - y_{\text{pred}}|$$

Characteristics:

Robust to outliers: Doesn't square errors, so large errors aren't disproportionately penalized

Same units as target: Directly interpretable

Less sensitive: Compared to MSE/RMSE for extreme values

MAE vs RMSE:

MAE: All errors weighted equally → use when outliers shouldn't dominate

RMSE: Large errors penalized more → use when large errors are particularly undesirable

Code:

```
pythondef mean_absolute_error(y_true, y_pred):
```

```
    mae = np.mean(np.abs(y_true - y_pred))
```

```
    return mae
```

Example Usage:

```
pythony_true = np.array([100, 150, 200, 250])
```

```
y_pred = np.array([110, 145, 205, 240])
```

```
mae = mean_absolute_error(y_true, y_pred)
```

On average, predictions are off by MAE units

Classification Metrics

These metrics are used to evaluate the performance of classification models.

5. Accuracy Score

accuracy_score

Accuracy measures the proportion of correct predictions out of total predictions.

Range:

Best value: 1.0 (100% correct)

Worst value: 0.0 (0% correct)

Range: [0, 1]

Mathematical Formula:

Accuracy = (Number of Correct Predictions) / (Total Predictions)

Accuracy = (TP + TN) / (TP + TN + FP + FN)

When to use:

Balanced datasets: When classes are roughly equal in size

Overall performance: When all types of errors are equally important

Limitations:

Misleading on imbalanced data: 95% accuracy might seem good, but if 95% of data is one class, a model predicting only that class achieves this

Doesn't distinguish error types: Doesn't tell you about false positives vs false negatives

Code:

```
pythondef accuracy_score(y_true, y_pred):
```

```
    correct_predictions = np.sum(y_true == y_pred)
```

```
    accuracy = correct_predictions / len(y_true)
```

return accuracy

Example Usage:

```
python_y_true = np.array([1, 0, 1, 1, 0, 1])
```

```
y_pred = np.array([1, 0, 1, 0, 0, 1])
```

```
accuracy = accuracy_score(y_true, y_pred)
```

5 out of 6 correct = 0.833 or 83.3%

6. Precision Score

precision_score

Precision measures the proportion of positive predictions that are actually correct. It answers: "Of all instances predicted as positive, how many were truly positive?"

Range:

Best value: 1.0 (all positive predictions are correct)

Worst value: 0.0 (no positive predictions are correct)

Range: [0, 1]

Mathematical Formula:

$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Where:

TP (True Positives): Correctly predicted positive cases

FP (False Positives): Incorrectly predicted as positive

When to use:

Cost of false positives is high: Email spam detection (don't want to mark legitimate emails as spam)

Focus on positive predictions: Medical tests where false alarms are costly

Code:

```
pythondef precision_score(y_true, y_pred):  
  
    tp = np.sum((y_true == 1) & (y_pred == 1))  
  
    fp = np.sum((y_true == 0) & (y_pred == 1))  
  
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0  
  
    return precision
```

Example Usage:

```
pythony_true = np.array([1, 0, 1, 1, 0, 1])  
  
y_pred = np.array([1, 1, 1, 0, 0, 1])  
  
precision = precision_score(y_true, y_pred)  
  
3 true positives out of 4 positive predictions = 0.75
```

7. Recall Score (Sensitivity)

recall_score

Recall measures the proportion of actual positive cases that were correctly identified. It answers: "Of all actual positive instances, how many did we catch?"

Range:

Best value: 1.0 (all positive cases found)

Worst value: 0.0 (no positive cases found)

Range: [0, 1]

Mathematical Formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Where:

TP (True Positives): Correctly predicted positive cases

FN (False Negatives): Missed positive cases (predicted as negative)

When to use:

Cost of false negatives is high: Disease detection (missing a sick patient is dangerous)

Find all positive cases: Fraud detection (want to catch all fraud, even if some false alarms)

Trade-off with Precision:

High Recall, Low Precision: Catches most positives but many false alarms

High Precision, Low Recall: Few false alarms but misses many positives

Code:

```
pythondef recall_score(y_true, y_pred):  
  
    tp = np.sum((y_true == 1) & (y_pred == 1))  
  
    fn = np.sum((y_true == 1) & (y_pred == 0))  
  
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0  
  
    return recall
```

Example Usage:

```
pythony_true = np.array([1, 0, 1, 1, 0, 1])  
  
y_pred = np.array([1, 1, 1, 0, 0, 1])  
  
recall = recall_score(y_true, y_pred)
```

3 out of 4 actual positives found = 0.75

8. F1 Score

f1_score

F1 Score is the harmonic mean of precision and recall. It provides a single metric that balances both precision and recall.

Range:

Best value: 1.0 (perfect precision and recall)

Worst value: 0.0

Range: [0, 1]

Mathematical Formula:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Why Harmonic Mean?

Harmonic mean punishes extreme values more than arithmetic mean

If either precision or recall is low, F1 will be low

Better represents balance than simple average

When to use:

Balanced measure: When you need to consider both precision and recall

Imbalanced datasets: More informative than accuracy

Single metric needed: When you want one number summarizing performance

Example:

Precision = 0.8, Recall = 0.6

$$\text{Arithmetic Mean} = (0.8 + 0.6) / 2 = 0.7$$

$$F1 \text{ Score} = 2 \times (0.8 \times 0.6) / (0.8 + 0.6) = 0.686$$

F1 is lower because it penalizes the imbalance.

Code:

```
pythondef f1_score(y_true, y_pred):  
  
    precision = precision_score(y_true, y_pred)  
  
    recall = recall_score(y_true, y_pred)  
  
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0  
  
    return f1
```

Example Usage:

```
pythony_true = np.array([1, 0, 1, 1, 0, 1])  
  
y_pred = np.array([1, 1, 1, 0, 0, 1])  
  
f1 = f1_score(y_true, y_pred)
```

Balances precision (0.75) and recall (0.75) = 0.75

9. Confusion Matrix

confusion_matrix

A confusion matrix is a table that visualizes the performance of a classification model by showing the counts of true positives, true negatives, false positives, and false negatives.

Structure:

Predicted

Negative Positive

Actual Neg | TN | FP |

Pos | FN | TP |

Where:

TN (True Negative): Correctly predicted as negative

FP (False Positive): Incorrectly predicted as positive (Type I Error)

FN (False Negative): Incorrectly predicted as negative (Type II Error)

TP (True Positive): Correctly predicted as positive

Why it's useful:

Complete picture: Shows all four types of predictions

Identifies error patterns: See which type of errors are more common

Basis for other metrics: Precision, recall, etc. are derived from it

Code:

```
pythondef confusion_matrix(y_true, y_pred):
```

```
    tp = np.sum((y_true == 1) & (y_pred == 1))
```

```
    tn = np.sum((y_true == 0) & (y_pred == 0))
```

```
    fp = np.sum((y_true == 0) & (y_pred == 1))
```

```
    fn = np.sum((y_true == 1) & (y_pred == 0))
```

```
    return np.array([[tn, fp],
```

```
                    [fn, tp]])
```

Example Usage:

```
pythony_true = np.array([1, 0, 1, 1, 0, 1])
```

```
y_pred = np.array([1, 1, 1, 0, 0, 1])
```

```
cm = confusion_matrix(y_true, y_pred)
```

Returns: **[[1, 1],**

[1, 3]]

TN=1, FP=1, FN=1, TP=3

Interpretation Example:

For a medical test:

TP: Sick patients correctly identified

TN: Healthy patients correctly identified

FP: Healthy patients wrongly identified as sick (false alarm)

FN: Sick patients wrongly identified as healthy (missed diagnosis)

10. Classification Report

classification_report

A classification report provides a comprehensive summary of all key classification metrics in one place.

Metrics Included:

Precision: Quality of positive predictions

Recall: Coverage of actual positive cases

F1-Score: Harmonic mean of precision and recall

Accuracy: Overall correctness

Purpose:

Quick overview: See all metrics at once

Model comparison: Easy to compare different models

Performance summary: Complete picture of classification performance

Code:

```
pythondef classification_report(y_true, y_pred):
```

```
precision = precision_score(y_true, y_pred)
```

```
recall = recall_score(y_true, y_pred)
```

```
f1 = f1_score(y_true, y_pred)
```

```
accuracy = accuracy_score(y_true, y_pred)
```

```
report = {
```

```
    'precision': precision,
```

```
    'recall': recall,
```

```
    'f1-score': f1,
```

```
    'accuracy': accuracy
```

```
}
```

```
return report
```

Example Usage:

```
pythony_true = np.array([1, 0, 1, 1, 0, 1])
```

```
y_pred = np.array([1, 1, 1, 0, 0, 1])
```

```
report = classification_report(y_true, y_pred)
```

Returns dictionary with all metrics

```
print(f"Precision: {report['precision']:.3f}")
```

```
print(f"Recall: {report['recall']:.3f}")
```

```
print(f"F1-Score: {report['f1-score']:.3f}")
```

```
print(f"Accuracy: {report['accuracy']:.3f}")
```

Loss Functions

These functions measure the difference between predicted and actual values during training.

11. Binary Cross-Entropy Loss

cross_entropy_loss

Binary cross-entropy (also called log loss) measures the performance of a binary classification model whose output is a probability between 0 and 1.

Range:

Best value: 0.0 (perfect predictions)

Worst value: ∞ (completely wrong predictions)

Lower is better

Mathematical Formula:

$$\text{BCE} = -(1/n) \times \sum [y \times \log(\hat{y}) + (1-y) \times \log(1-\hat{y})]$$

Where:

y = true label (0 or 1)

$\hat{y}$ = predicted probability

Why it's used:

Penalizes confidence: Wrong confident predictions are heavily penalized

Smooth gradient: Provides good gradient for optimization

Probabilistic interpretation: Related to maximum likelihood estimation

Clipping:

The function clips predictions to avoid $\log(0)$ which would be undefined:

```
pythonepsilon = 1e-15
```

```
y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
```

Code:

```
pythondef cross_entropy_loss(y_true, y_pred):
```

```
    epsilon = 1e-15
```

```
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
```

```
    ce_loss = -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
```

```
    return ce_loss
```

Example Usage:

```
pythony_true = np.array([1, 0, 1, 0])
```

```
y_pred = np.array([0.9, 0.1, 0.8, 0.2]) # Probabilities
```

```
loss = cross_entropy_loss(y_true, y_pred)
```

Low loss indicates good predictions

Interpretation:

$y=1, \hat{y}=0.9$: Loss = $-\log(0.9) = 0.105$ (good prediction)

$y=1, \hat{y}=0.1$: Loss = $-\log(0.1) = 2.303$ (bad prediction)

Wrong confident predictions result in very high loss

12. Categorical Cross-Entropy Loss

categorical_crossentropy_loss

Categorical cross-entropy measures the performance of a multi-class classification model whose output is a probability distribution over multiple classes.

Range:

Best value: 0.0 (perfect predictions)

Worst value: ∞ (completely wrong predictions)

Lower is better

Mathematical Formula:

$$\text{CCE} = -(1/n) \times \sum_i \sum_j y_{i,j} \times \log(\hat{y}_{i,j})$$

Where:

$y_{i,j}$ = true one-hot encoded label for sample i , class j

$\hat{y}_{i,j}$ = predicted probability for sample i , class j

Use Case:

Multi-class classification: When there are more than 2 classes

Softmax output: Typically used with softmax activation in output layer

One-hot encoded labels: Requires labels in one-hot format

Example:

True label: [0, 1, 0] (class 1)

Prediction: [0.1, 0.8, 0.1]

$$\text{Loss} = -(0 \times \log(0.1) + 1 \times \log(0.8) + 0 \times \log(0.1)) = -\log(0.8) = 0.223$$

Code:

```
python def categorical_cross_entropy_loss(y_true, y_pred):
```

```
    epsilon = 1e-15
```

```
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
```

```
cce_loss = -np.mean(np.sum(y_true * np.log(y_pred), axis=1))
```

```
return cce_loss
```

Example Usage:

```
python# 3 samples, 3 classes
```

```
y_true = np.array([[1, 0, 0], # Class 0
```

```
[0, 1, 0], # Class 1
```

```
[0, 0, 1]]) # Class 2
```

```
y_pred = np.array([[0.7, 0.2, 0.1],
```

```
[0.1, 0.8, 0.1],
```

```
[0.2, 0.2, 0.6]])
```

```
loss = categorical_cross_entropy_loss(y_true, y_pred)
```

Low loss indicates good predictions

Difference from Binary Cross-Entropy:

Binary: 2 classes, single probability output

Categorical: Multiple classes, probability distribution output

Summary

This documentation covers seven fundamental machine learning algorithms implemented from scratch:

1. **Linear Regression** - For continuous value prediction
2. **Polynomial Regression** - For non-linear relationships

3. **Logistic Regression** - For binary classification
4. **Neural Network** - For complex pattern recognition
5. **Decision Tree** - For interpretable classification
6. **K-Nearest Neighbors** - For instance-based learning
7. **K-Means Clustering** - For unsupervised grouping
8. **train_test_split**: Divides data for proper model evaluation
9. **z_score_normalization**: Standardizes features to same scale
10. **one_hot_encode**: Converts categorical labels for neural networks
11. **pca**: Reduces dimensionality while preserving information

Each implementation includes training methods, prediction capabilities, and comprehensive evaluation metrics.